

Table of Contents

SyncBoard — Product Requirements Document (PRD)

Real-Time Collaborative Kanban Platform

Table of Contents

1. Executive Summary

2. Problem Statement & Vision

Problem Statement

Vision

Guiding Principles

3. Goals & Success Metrics

Core Objectives

Success Criteria

Metrics to Track During Development (self-evaluation, not user-facing analytics)

4. Target Users & Personas

5. Scope

In Scope (v1)

Explicitly Out of Scope (v1)

Candidate for v2 (explicitly deferred, not forgotten)

6. System Architecture

6.1 Architectural Style

6.2 High-Level Component Map

6.3 The Two Data Paths

6.4 Request/Event Flow Pattern

6.5 Connection Topology

7. Technology Stack & Rationale

8. Data Model

8.1 Entities

8.2 Relationships (Cardinality)

8.3 Design Notes Worth Preserving

9. Feature Modules (Detailed Specs)

9.1 Module: Authentication

9.2 Module: Workspace & Board Management

9.3 Module: Presence Tracking

9.4 Module: Real-Time Card Editing (Soft Lock + Optimistic Concurrency)

9.5 Module: Live Typing Indicators

9.6 Module: Kanban Board (Drag & Drop)

9.7 Module: Notifications

- 9.8 Module: Comments
 - 9.9 Module: Activity Feed
 - 10. Detailed User Workflows
 - 10.1 Registration & First Login
 - 10.2 Creating a Workspace and Inviting Members
 - 10.3 Opening a Board (Initial Sync)
 - 10.4 Moving a Card (End-to-End)
 - 10.5 Editing a Card Someone Else Is Also Viewing
 - 10.6 Reconnection After a Dropped Connection
 - 10.7 Notification Lifecycle
 - 11. Real-Time Communication Design
 - 11.1 Topic Structure
 - 11.2 Event Catalog (Conceptual, Not Payload Schemas)
 - 11.3 Why STOMP Instead of Raw WebSocket
 - 12. REST API Surface
 - 13. Roles & Permissions
 - 14. Non-Functional Requirements
 - 15. UI/UX Page Specifications
 - 16. Edge Cases & Failure Handling
 - 17. Security Considerations
 - 18. Development Roadmap
 - 19. Testing Strategy
 - 20. Deployment & Infrastructure
 - 21. Future Enhancements (Post-v1, Resume-Worthy Additions)
 - 22. Glossary
-

SyncBoard — Product Requirements Document (PRD)

Real-Time Collaborative Kanban Platform

Version: 1.0

Status: Draft for personal development

Author: (You)

Purpose: Complete reference document to build SyncBoard end-to-end without needing to re-derive scope, architecture, or workflow decisions mid-project.

Note: I've used the name "SyncBoard" throughout since it communicates the real-time sync angle clearly — rename freely if you land on something else. This document intentionally contains **no code**. It is meant to be the blueprint you build from.

Table of Contents

1. Executive Summary
 2. Problem Statement & Vision
 3. Goals & Success Metrics
 4. Target Users & Personas
 5. Scope (In / Out)
 6. System Architecture
 7. Technology Stack & Rationale
 8. Data Model
 9. Feature Modules (Detailed Specs)
 10. Detailed User Workflows
 11. Real-Time Communication Design
 12. REST API Surface
 13. Roles & Permissions
 14. Non-Functional Requirements
 15. UI/UX Page Specifications
 16. Edge Cases & Failure Handling
 17. Security Considerations
 18. Development Roadmap
 19. Testing Strategy
 20. Deployment & Infrastructure
 21. Future Enhancements
 22. Glossary
-

1. Executive Summary

SyncBoard is a lightweight, real-time collaborative Kanban board — similar in spirit to Trello, but built specifically to *demonstrate* how real-time multi-user systems work under the hood rather than to compete on features. Every module exists to showcase a distinct engineering concept: authentication, WebSocket-based live sync, presence tracking, optimistic concurrency, soft-locking, and event-driven notifications.

The defining trait of SyncBoard is that **the board state is never stale**. If two people have the board open, every drag, edit, comment, and status change appears instantly on both screens without a manual refresh — and the system is honest about *who* is doing what, in real time.

This document lays out what to build, why each piece exists, how the pieces interact, and in what order to build them — without prescribing implementation code, so you retain full ownership of the actual engineering decisions.

2. Problem Statement & Vision

Problem Statement

Teams — especially small ones, students, or hackathon groups — need a shared task board. Existing tools fall into two categories:

- **Heavyweight, enterprise tools** (Jira, Asana): overloaded with configuration, permissions, and workflow complexity that a small team doesn't need.
- **Simple tools** (basic Trello clones): usually built as CRUD apps with a refresh button, not as genuinely real-time systems. They don't actually demonstrate live collaboration — they demonstrate a database with a nice UI.

There's a gap for a tool that is *simple in scope* but *deep in real-time engineering* — where the actual technical challenge is synchronization, concurrency, and presence, not feature count.

Vision

Build a Kanban board where the board itself feels "alive": you can see teammates' cursors of attention (who's viewing what), see edits happen as they happen, see typing indicators mid-comment, and never worry about silently overwriting someone else's change. The product succeeds if, in a live demo, two browser windows side-by-side feel like a single shared surface rather than two independent clients polling a server.

Guiding Principles

- **Soft-lock, never hard-lock.** Never prevent someone from acting; instead, make it transparent who else is acting, and let optimistic concurrency resolve genuine conflicts.
 - **Every state change is an event.** If it changes on screen, it travels through a WebSocket event, not a page reload.
 - **Single-server first.** Resist the urge to add Kafka, microservices, or multi-node complexity. The goal is to master real-time mechanics on a single well-designed server before ever thinking about distributed scale.
 - **Transparency over restriction.** Presence, editing indicators, and activity logs exist to make collaboration visible, not to gate it.
-

3. Goals & Success Metrics

Core Objectives

The project should give you hands-on, defensible experience in:

- Spring Boot application structure and layered architecture
- Spring Security with JWT-based stateless authentication
- STOMP-over-WebSocket real-time messaging (with SockJS fallback)
- React application structure, state management, and drag-and-drop UX
- Optimistic UI updates and rollback handling
- Presence tracking and heartbeat-based liveness detection
- Database design for a moderately relational domain
- Session/token lifecycle management (access + refresh tokens)

Success Criteria

A build is "done" for a given phase when:

- Two browser sessions (e.g., a normal window and an incognito window), logged in as two different users, viewing the same board, see **every** change the other makes within ~200ms, with no manual refresh.

- Closing one browser tab causes the other user's presence list to update within a few seconds (via heartbeat timeout), not instantly relying on a clean disconnect event alone (since real networks don't always send one).
- Editing the same card from two tabs never silently loses one person's changes — the system either merges, warns, or rejects with a clear reason.

Metrics to Track During Development (self-evaluation, not user-facing analytics)

- Round-trip latency from action (e.g., drag card) to remote reflection.
 - Reconnect time after simulated network drop (e.g., via browser dev tools "offline" toggle).
 - Number of WebSocket topics a single client subscribes to at once, and whether that scales sensibly per board.
-

4. Target Users & Personas

Although this is a portfolio/learning project, designing around real personas keeps feature decisions grounded.

Persona 1 — "Harshal", a hackathon team lead

Needs to spin up a board in seconds, invite 3–4 teammates, and see instantly who's working on what during a 24-hour sprint. Cares about speed and clarity, not configuration.

Persona 2 — "Rahul", a small startup engineer

Uses the board daily across a 2-week sprint. Cares about not losing work when two people touch the same card, and wants a reliable activity log to reconstruct "what happened" after the fact.

Persona 3 — "A recruiter or interviewer"

Opens the deployed app, creates two browser sessions, and evaluates whether the "real-time" claim is actually real. This persona cares about correctness under stress (rapid actions, disconnects) more than visual polish.

5. Scope

In Scope (v1)

- Email/password registration and login with JWT access + refresh tokens
- Workspaces containing multiple boards
- Boards containing ordered columns, columns containing ordered cards
- Drag-and-drop card movement with real-time broadcast
- Presence (online/away/offline) with heartbeat
- Soft-lock "currently editing" indicators per card
- Live typing indicators inside comment threads
- Comments per card
- Notifications (in-app, real-time) for assignment, mentions, completion
- Activity feed per workspace
- Reconnect logic with state resynchronization after a dropped connection

Explicitly Out of Scope (v1)

- Kafka, message queues, or distributed brokers — single Spring Boot instance is sufficient
- Microservices — this is a modular monolith
- Mobile native apps
- Third-party integrations (Slack, GitHub, email digests)
- File attachments beyond simple image/link embedding in comments
- Billing/payments (this isn't a commercial product)

Candidate for v2 (explicitly deferred, not forgotten)

- Redis-backed presence/pub-sub if you outgrow single-instance WebSocket session storage
 - Role-based granular permissions beyond Owner/Admin/Member
 - Full-text search across cards and comments
 - Keyboard-shortcut power-user mode
-

6. System Architecture

6.1 Architectural Style

SyncBoard is a **modular monolith**: one Spring Boot backend, one React frontend, one PostgreSQL database. Modularity is enforced through package/module boundaries inside the monolith (e.g., auth, workspace, board, presence, notification, activity), not through network boundaries between services. This gives you clean separation of concerns without the operational overhead of distributed systems — which is the correct trade-off for a single-developer or small-team project.

6.2 High-Level Component Map

Describe the system as five cooperating layers:

1. **Client Layer (React SPA)** — renders the UI, holds local/optimistic state, opens one persistent WebSocket connection per session, and issues REST calls for anything that isn't inherently "live" (e.g., initial page load, historical data fetch).
2. **API Gateway Layer (Spring Boot Controllers)** — exposes REST endpoints for CRUD-style operations (create board, fetch cards, etc.) and a STOMP endpoint for real-time messaging. This layer authenticates every request via JWT.
3. **Real-Time Messaging Layer (Spring WebSocket + STOMP broker)** — routes events between clients using topic-based publish/subscribe. This is the heart of the "live" experience.
4. **Domain/Service Layer** — contains business logic: what happens when a card moves, who should be notified, how presence timeouts are computed, how optimistic concurrency conflicts are detected.
5. **Persistence Layer (Spring Data JPA + PostgreSQL)** — the source of truth. Every real-time event that represents a durable state change is persisted here; ephemeral events (typing indicators, "currently viewing") are not persisted, only broadcast.

6.3 The Two Data Paths

It's important to mentally separate SyncBoard's two distinct data paths, because they have different guarantees:

- **Durable path (REST + DB):** Used for anything that must survive a server restart — user accounts, boards, cards, comments, notifications, activity log entries. Always goes through the service layer, always persisted, always followed by a broadcast so other clients learn about it.
- **Ephemeral path (WebSocket only):** Used for anything transient — "user is typing," "user is currently viewing this card," heartbeat pings. Never touches the database (with the arguable exception of last-seen timestamps). If the server restarts, this state is simply gone and rebuilt as clients reconnect.

Keeping this distinction explicit prevents a common mistake: trying to persist every micro-event, which bloats the database and adds needless latency to things that don't need durability.

6.4 Request/Event Flow Pattern

Every meaningful action in the app should conceptually follow this same shape, regardless of feature:

1. Client performs a local optimistic update (e.g., immediately shows the card in its new column).
2. Client sends the actual request — either a REST call (for durable changes) or a STOMP message (for ephemeral ones).
3. Server validates, applies business rules, and (for durable changes) persists to the database.
4. Server broadcasts the resulting canonical event to every subscriber of the relevant topic — including the originating client.
5. Every client (including the originator) reconciles its local state against the canonical broadcast. If the optimistic update matches, nothing visibly changes. If it doesn't (e.g., the server rejected the move due to a stale version), the client rolls back and shows a brief correction.

This "server is the single source of truth, client always reconciles against the broadcast" pattern is what makes multi-client consistency reliable, and it's the single most important architectural decision in this project.

6.5 Connection Topology

Each logged-in client opens exactly **one** WebSocket connection (via SockJS for browser compatibility, with STOMP as the messaging protocol on top). Through that single connection, the client subscribes to multiple topics simultaneously (e.g., its personal notification topic, the presence topic, and the topic for whichever board is currently open). This avoids the anti-pattern of one socket per feature.

7. Technology Stack & Rationale

Layer	Choice	Why
Frontend framework	React + React Router	Component model suits a board of nested, independently-updating widgets (cards, columns, presence list)

Layer	Choice	Why
Styling	Tailwind CSS	Fast iteration without hand-rolled CSS files per component
Drag & drop	React DnD	Purpose-built for exactly this interaction pattern; handles drag state, drop targets, and reordering logic
Real-time client	SockJS + STOMP.js	SockJS provides fallback transports if raw WebSocket is blocked; STOMP gives you topic-based pub/sub semantics instead of raw message parsing
HTTP client	Axios	Interceptors make JWT attachment and refresh-token retry logic centralized and clean
Backend framework	Spring Boot	Batteries-included: security, WebSocket, JPA, validation all integrate natively
Security	Spring Security + JWT	Stateless auth fits a horizontally-simple, single-instance deployment; no server-side session store needed for REST
Real-time server	Spring WebSocket + STOMP broker	First-class Spring integration; topic subscriptions map cleanly to board/workspace/user scopes
ORM	Spring Data JPA	Reduces boilerplate for a fairly standard relational schema
Database	PostgreSQL	Strong relational integrity for the workspace → board → column → card hierarchy; mature, well-documented
Containerization	Docker + Docker Compose	One-command local environment (DB + backend + frontend)
Optional (v2)	Redis	Only needed if you outgrow in-memory presence tracking or want pub/sub across multiple backend instances — not needed for v1's single-instance design
Optional (v2)	Nginx	Reverse proxy / static file serving once you deploy beyond local Docker Compose

Why not Kafka or microservices for v1: the entire value of this project is demonstrating that you understand real-time synchronization, concurrency control, and presence *correctly* on a single, well-modeled server. Adding a message broker or splitting services introduces distributed-systems problems (network partitions, eventual consistency, service discovery) that are irrelevant to what you're trying to prove, and they dilute the portfolio story rather than strengthening it.

8. Data Model

Below is the conceptual entity model — described in terms of what each table represents and how it relates to others, not schema syntax.

8.1 Entities

User

Represents an account. Holds identity (name, email), authentication material (hashed password), profile (avatar), and current presence state (status, last-seen timestamp). Presence fields are denormalized onto the user for fast lookup, even though presence is also broadcast live.

Workspace

The top-level container. Has one owner and many members. A user can belong to multiple

workspaces. All boards live inside exactly one workspace.

WorkspaceMember

A join entity between User and Workspace, carrying the member's role (Owner, Admin, Member) within that specific workspace. This is what makes permissions workspace-scoped rather than global.

Board

Belongs to a workspace. Has an ordered position (for workspaces with multiple boards) and a title. Contains columns.

Column

Belongs to a board. Has a title and an ordered position (Todo, Doing, Testing, Done, etc., but user-renameable). Contains cards.

Card

Belongs to a column. This is the most feature-dense entity: title, description, priority, assignee, deadline, ordered position within the column, and — critically — a **version number** for optimistic concurrency control (see §9.4). Cards are the unit that gets soft-locked during editing and the unit that drag-and-drop moves between columns.

Comment

Belongs to a card and a user (the author). Holds message content and a timestamp. Supports mentions (a reference to another user) which feed the notification system.

Notification

Belongs to a receiving user. Holds a message, a read/unread flag, and a timestamp. Generated by the system in response to events like assignment, mention, or completion — never created directly by a user action alone.

Activity

Belongs to a workspace, records a user and an action description with a timestamp. This is the durable audit trail — distinct from ephemeral presence/typing events, which are never persisted.

8.2 Relationships (Cardinality)

- One **User** ↔ many **WorkspaceMember** ↔ many **Workspace** (many-to-many, through the join entity)
- One **Workspace** → many **Board**
- One **Board** → many **Column**
- One **Column** → many **Card**
- One **Card** → many **Comment**
- One **User** → many **Notification** (as receiver)
- One **Workspace** → many **Activity**

8.3 Design Notes Worth Preserving

- **Position fields** (on Board, Column, and Card) should be modeled as a sortable numeric value (not just array index) so reordering one item doesn't require rewriting every sibling's position. This is a classic drag-and-drop persistence problem worth solving deliberately rather than accidentally.
- **Card.version** exists solely to support optimistic locking — every update to a card should check the version it was read at and increment it on success, rejecting stale writes. This is described in detail in §9.4.
- **last_seen** on User is the fallback presence signal; it's what lets you show "Last seen 5 minutes ago" for offline users, distinct from the live presence broadcast used while

someone is actually connected.

9. Feature Modules (Detailed Specs)

9.1 Module: Authentication

Purpose: Establish identity securely without server-side session state, so the backend can remain stateless and horizontally simple.

Behavior:

- Registration collects name, email, password; password is hashed before storage, never stored or logged in plaintext.
- Login issues a short-lived **access token** (used on every REST call and to authenticate the WebSocket handshake) and a longer-lived **refresh token** (used only to silently obtain a new access token without forcing re-login).
- Logout should invalidate the refresh token server-side (e.g., via a blacklist or rotation record) so a stolen refresh token can't be used indefinitely after logout.
- The access token should be attached to the WebSocket handshake (commonly via a query parameter or a STOMP CONNECT header) since browsers don't allow custom headers on the initial WebSocket upgrade request the same way they do for REST.

Why it matters for the portfolio story: stateless JWT auth combined with a stateful WebSocket connection is a genuinely interesting engineering seam — the WebSocket connection is inherently stateful (it stays open), while your REST auth model is stateless. Handling that seam correctly (e.g., what happens when an access token expires mid-connection) is a strong talking point.

9.2 Module: Workspace & Board Management

Purpose: Organize boards into team-scoped containers with membership and roles.

Behavior:

- A user creates a workspace and automatically becomes its Owner.
- The Owner (or Admin) invites other users, who become Members.
- A workspace contains multiple boards (not just one shared board) — this was a deliberate improvement over a single-shared-board design, since real teams run several concurrent efforts.
- Deleting a workspace, board, or column should cascade sensibly (deleting a board removes its columns and cards) but should be a deliberate, confirmed action given the destructive blast radius.

9.3 Module: Presence Tracking

Purpose: Make it visually obvious, at all times, who is currently around and how active they are.

States to track: Online (actively connected), Idle (connected but no interaction for a short window, e.g., 2 minutes), Away (no interaction for a longer window, e.g., 5 minutes), Offline (not connected).

Mechanism:

- On WebSocket connect, the server marks the user Online and broadcasts an updated presence list to everyone in the relevant workspace/board.
- A **heartbeat** (a lightweight ping sent by the client on a fixed interval, e.g., every 15–30 seconds) is the primary liveness signal — relying solely on the WebSocket "disconnect" event is not sufficient, because real networks can drop silently (phone sleeps, wifi drops) without a clean close frame ever arriving.
- The server tracks the last heartbeat timestamp per connection. If a heartbeat is missed for longer than a defined grace period, the server proactively marks that user Offline and broadcasts the change — this is what makes presence trustworthy even under messy real-world network conditions.
- Idle/Away transitions can be derived client-side from user input inactivity (mouse/keyboard) and communicated to the server as a status update, rather than the server trying to infer "idle" from network activity alone.

Why heartbeat-based detection matters: it's the difference between a demo that only works when you cleanly close a tab, and a system that behaves correctly when someone's laptop lid snaps shut. This is one of the more resume-worthy details in the whole project — be ready to explain it in an interview.

9.4 Module: Real-Time Card Editing (Soft Lock + Optimistic Concurrency)

Purpose: Let multiple people work on the same board simultaneously without silent data loss, while never fully blocking anyone.

Two complementary mechanisms work together here — don't conflate them:

a) Soft-lock (a UX signal, not an enforcement mechanism):

When a user opens a card for editing, the client broadcasts an "editing started" event. Every other client viewing that card shows a small indicator (e.g., "Currently edited by Harshal"). This is purely advisory — it does not prevent a second user from also opening and editing the same card. Its only job is to reduce the *chance* of conflict by making it visible, not to guarantee it.

b) Optimistic concurrency control (the actual enforcement mechanism):

Every card carries a version number. When a client submits an edit, it includes the version it started from. The server checks that version against the current database value:

- If they match, the update is accepted, the version increments, and the change is persisted and broadcast.
- If they don't match (someone else saved a change in between), the server rejects the update and returns the current canonical state. The client then needs a clear UX response — typically: show the newer version, and let the user decide whether to reapply their change on top of it, rather than silently overwriting.

Why both are needed: the soft-lock indicator reduces how often conflicts happen (social prevention); the version check guarantees that when a conflict *does* happen anyway, no one's work silently disappears (technical guarantee). A production-quality real-time system needs both — relying on only the visual indicator is not actually safe, and relying on only version checks without any visible signal creates a frustrating experience of edits "randomly" getting rejected.

9.5 Module: Live Typing Indicators

Purpose: Give comment threads the same "alive" feeling as chat apps.

Behavior: While a user has focus in a card's comment input, the client periodically broadcasts a lightweight "user is typing" ephemeral event, scoped to that card. Other clients

viewing the same card show a transient "Harshal is typing..." line that disappears automatically after a short timeout (e.g., 3–5 seconds) if no further typing events arrive — rather than waiting for an explicit "stopped typing" event, which is easy to miss if a tab is closed abruptly.

9.6 Module: Kanban Board (Drag & Drop)

Purpose: The core interaction surface — moving cards between columns, and reordering within a column.

Behavior:

- Drag-and-drop is handled entirely client-side for the visual interaction; on drop, the client immediately reflects the new position optimistically, then sends the move to the server.
- The server recalculates the card's position value (see §8.3 on sortable position fields), persists the change, increments the card's version, and broadcasts the canonical move to every subscriber of that board's topic.
- If the optimistic client-side position and the server's canonical position differ (rare, but possible under concurrent moves), the client should smoothly correct to the canonical position rather than jarringly snapping.

9.7 Module: Notifications

Purpose: Ensure users learn about relevant events even when not looking directly at the affected card.

Trigger events: card assignment, being @mentioned in a comment, a card you're assigned to being marked complete, and other workspace-relevant events you choose to add.

Behavior: Notifications are generated server-side as a *consequence* of another action (never created directly by a raw client request) — this keeps notification logic centralized and consistent. Each notification is persisted (so a user can see their notification history even if they were offline when it happened) and also broadcast live to the affected user's personal notification topic if they're currently connected.

9.8 Module: Comments

Purpose: Threaded discussion scoped to a single card.

Behavior: Comments support plain text, embedded links, and @mentions of other workspace members. Posting a comment persists it, broadcasts it live to everyone viewing that card, and (if it contains a mention) triggers the notification module for the mentioned user.

9.9 Module: Activity Feed

Purpose: A durable, chronological audit trail of everything meaningful that happened in a workspace.

Behavior: Every durable state change (card moved, card created, comment added, priority changed, member joined) generates an activity entry — distinct from ephemeral events like typing or presence, which are intentionally *not* logged here. The activity feed is what someone would read to answer "what happened while I was away," so it should read like a clean, human-readable log rather than raw event data.

10. Detailed User Workflows

These are the step-by-step flows worth designing (and later testing against) explicitly.

10.1 Registration & First Login

1. User submits registration form → server validates uniqueness of email, hashes password, creates the User record.
2. User logs in → server verifies credentials, issues access + refresh tokens.
3. Client stores tokens (access token in memory is safer than localStorage; refresh token can be in an httpOnly cookie if your architecture supports it) and redirects to the Dashboard.
4. Client opens the WebSocket connection, authenticating the handshake with the access token, and subscribes to the user's personal notification topic and presence topic.

10.2 Creating a Workspace and Inviting Members

1. User creates a workspace → becomes Owner automatically.
2. Owner invites a teammate by email → an invitation record (or direct membership, depending on how strict you want onboarding to be) is created.
3. Invited user, upon accepting, becomes a WorkspaceMember with role "Member."
4. All current members' clients receive a "member joined" broadcast on the workspace topic and the activity feed updates.

10.3 Opening a Board (Initial Sync)

1. Client requests the board's full current state via REST (columns, cards, positions) — this is the "cold start" snapshot.
2. Client subscribes to the board's WebSocket topic for ongoing live updates.
3. Client separately subscribes to the presence topic to render the online members list.
4. From this point forward, the client should never need to re-fetch the board via REST unless it detects it missed events (e.g., after a reconnect — see §10.6).

10.4 Moving a Card (End-to-End)

1. User drags a card from "Todo" to "Doing."
2. Client immediately re-renders the card in "Doing" (optimistic).
3. Client sends the move (source column, target column, target position, current known version) to the server.
4. Server validates the move, recalculates position values, increments the card's version, persists, and broadcasts the canonical CARD_UPDATED event to the board's topic.
5. Every subscribed client (including the originator) applies the broadcast. The originator's optimistic state already matches, so nothing visibly changes for them; every other client sees the card move live.

10.5 Editing a Card Someone Else Is Also Viewing

1. User A opens the card detail drawer → client broadcasts CARD_EDITING_STARTED.
2. User B, also viewing the card, sees "Currently edited by A."
3. User B opens the same card anyway (soft-lock allows this) and makes a change, saving first.
4. Server accepts B's change (version matches), increments version, broadcasts the update.

5. User A, still editing, tries to save. Server rejects because A's held version is now stale.
6. Client shows A a clear message: someone else's change was saved first, here's the current version — decide whether to reapply your edit on top of it.

10.6 Reconnection After a Dropped Connection

1. Client detects the WebSocket has disconnected (either explicitly or via a failed heartbeat acknowledgment).
2. Client attempts reconnection with backoff (e.g., retry after 1s, 2s, 5s, capped).
3. On successful reconnect, rather than assuming no events were missed, the client re-fetches the current board state via REST (a full resync) and re-subscribes to all relevant topics.
4. This "reconnect = resync" rule is simpler and more robust than trying to replay a missed-event log, and is the right trade-off for this project's scale.

10.7 Notification Lifecycle

1. Some triggering action occurs (e.g., User B assigns a card to User A).
 2. Server creates a persisted Notification record for User A and, if User A is currently connected, broadcasts it live to their personal notification topic.
 3. User A's client shows a toast/badge update immediately if connected, or sees it in their notification panel on next login if not.
 4. User A marks it read (individually or via "mark all read") → server updates the read flag.
-

11. Real-Time Communication Design

11.1 Topic Structure

Design WebSocket topics around **scope**, so clients only subscribe to what's relevant to what they're currently viewing:

- A per-workspace topic — for workspace-level events like membership changes and the activity feed.
- A per-board topic — for column/card structural changes and moves. Clients subscribe only while that board is open.
- A per-card topic (optional, or you can fold this into the board topic) — for fine-grained editing/typing indicators, to avoid every board-viewer receiving every keystroke-adjacent event for cards they're not looking at.
- A presence topic — global or per-workspace, for online/away/offline state.
- A per-user personal topic — for private notifications meant only for that individual.

11.2 Event Catalog (Conceptual, Not Payload Schemas)

Event	Direction	Scope	Persisted?	Purpose
User connected	Client → Server	Connection	No	Marks presence online, triggers broadcast
Online users updated	Server → Clients	Presence topic	No	Refreshes the presence list
Card moved	Client → Server	Board topic	Yes	Requests a move; server validates & persists

Event	Direction	Scope	Persisted?	Purpose
Card updated	Server → Clients	Board topic	—	Canonical broadcast after any card change
Card editing started	Client → Server → Clients	Card/board topic	No	Soft-lock indicator
Card editing stopped	Client → Server → Clients	Card/board topic	No	Clears soft-lock indicator
User typing	Client → Server → Clients	Card topic	No	Transient typing indicator
New comment	Client → Server	Card topic	Yes	Persists and broadcasts a comment
Comment added	Server → Clients	Card/board topic	—	Canonical broadcast after comment persisted
Notification created	Server → Client	Personal topic	Yes	Delivers a real-time notification
Heartbeat ping/pong	Client ↔ Server	Connection	No	Liveness detection for presence

11.3 Why STOMP Instead of Raw WebSocket

Raw WebSocket gives you a single unstructured message stream per connection — you'd have to invent your own routing/subscription scheme on top of it. STOMP gives you destinations (topics) and a pub/sub model natively, which maps directly onto "a client cares about board X, card Y, and its own notifications" without you reinventing that routing layer yourself. This is the main reason it's worth the extra dependency over raw WebSocket for a project of this shape.

12. REST API Surface

Described by resource and intent, not exact routes/payloads (you'll design those as you build):

Auth: register, login, logout, refresh access token.

Workspace: create workspace, list my workspaces, invite member, list members, change a member's role, remove a member, leave a workspace.

Board: create board (within a workspace), list boards (within a workspace), fetch a single board's full state (columns + cards, for cold-start sync), rename/reorder a board, delete a board.

Column: create column, rename column, reorder columns, delete column.

Card: create card, fetch a single card's full detail (including comments), update card fields, move/reorder card (may overlap with the WebSocket path — decide whether moves go exclusively over WebSocket or are dual-pathed for reliability), delete card.

Comment: post comment, fetch comments for a card.

Notification: fetch my notifications, mark one/all as read.

Activity: fetch recent activity for a workspace (paginated).

Design principle: REST is for *fetching state* and for *durable writes that don't need sub-second propagation urgency* (e.g., renaming a board). WebSocket/STOMP is for anything where the "live" feeling matters or where ephemeral state is involved. Card moves and comments sit at the boundary — a common and reasonable pattern is: client sends the write over STOMP directly (since it's already connected and it's on the critical "feels live" path), and the server persists it exactly as it would a REST write, then broadcasts. REST endpoints for the same resources still exist for the initial full-state fetch.

13. Roles & Permissions

Role	Can Do
Owner	Everything an Admin can do, plus: delete the workspace, transfer ownership
Admin	Invite/remove members, change member roles (except Owner), create/delete boards, manage columns
Member	Create/edit/move/delete cards, comment, view activity feed and presence — cannot manage membership or delete boards

Keep permission checks in the service layer (not just hidden in the UI) since a determined client could otherwise bypass UI-only restrictions by calling the API directly.

14. Non-Functional Requirements

Category	Requirement
Latency	Under ~200ms from action to remote reflection under normal conditions
Concurrent users	Should comfortably handle 100+ concurrent connections on a single instance for this project's purposes
Reconnection	Automatic, with backoff, and a full state resync on reconnect (see §10.6)
Security	All state-changing requests authenticated via JWT; WebSocket handshake authenticated the same way
Availability	Not a formal SLA for a personal project, but design as if 99% mattered — i.e., don't let one bad connection crash the shared broker
Scalability	Single-server architecture is an explicit, deliberate v1 constraint — not a limitation to apologize for

15. UI/UX Page Specifications

Login / Register — minimal forms; register additionally collects display name and optional avatar.

Dashboard — landing page after login. Shows: recent boards across all workspaces, an online-members widget, a notifications summary, and recent activity across workspaces the user belongs to.

Workspace View — lists boards within the workspace, membership list with roles, an "invite member" action for Owners/Admins.

Board View — the core screen. Layout: a header showing the board title and the live presence list (avatars with status dots); the column/card grid itself, drag-and-drop enabled; a

collapsible side panel for the activity feed; a notification bell in the global nav.

Card Detail Drawer — opens over the board (not a full page navigation, to preserve board context). Shows full card fields, the comment thread with live typing indicators, the "currently edited by X" banner when relevant, and an edit history hint if you choose to surface version conflicts explicitly.

Profile / Settings — basic account info, avatar, and presence-related preferences (e.g., how long until "Away" triggers, if you choose to make that configurable).

Notification Panel — a dropdown or dedicated page listing notifications, unread-first, with mark-as-read controls.

16. Edge Cases & Failure Handling

Deliberately design for these rather than discovering them late:

- **Two users move the same card at nearly the same instant.** Resolved by the version-based optimistic concurrency check (§9.4) — the second write is rejected and reconciled, not silently overwritten.
 - **A user's connection drops mid-edit.** Their soft-lock indicator should not persist forever on other clients — tie the "currently editing" indicator to the same heartbeat/liveness mechanism as presence, so it clears if the editor's connection goes stale, not only on an explicit "stopped editing" event.
 - **A user deletes a card another user currently has open.** The viewing client should receive a clear "this card was deleted" event and gracefully close the drawer rather than erroring on a missing resource.
 - **Rapid repeated drags (a user frantically reordering cards).** The server should treat each move as authoritative in arrival order; the client shouldn't need special debouncing beyond normal UI responsiveness, since the server is the final arbiter of position.
 - **A workspace Owner removes a member who is currently connected and viewing the board.** That user's session should be gracefully revoked — their next action should be met with an authorization failure that the client interprets as "you've lost access," not a generic error.
 - **Refresh token expires while a user is mid-session.** The access-token refresh should happen transparently in the background (via an Axios interceptor pattern, conceptually) before the user notices — only force a full logout if the refresh token itself has expired or been revoked.
-

17. Security Considerations

- Passwords hashed with a strong, slow algorithm (e.g., bcrypt/argon2-family) — never reversible encryption.
- Access tokens short-lived (minutes to low hours); refresh tokens longer-lived but revocable server-side (so logout is meaningful).
- WebSocket handshake must be authenticated — don't allow an unauthenticated socket to subscribe to any topic.
- Authorization checks belong in the service layer, scoped per-workspace (a user's role in Workspace A should never grant privileges in Workspace B).
- Validate that a user is actually a member of a workspace/board before allowing them to subscribe to its topics or act on its resources — topic names should not be guessable/enumerable as a substitute for real authorization.

- Sanitize comment content to prevent stored XSS, since comments are rendered back to other users' browsers.
-

18. Development Roadmap

A phased build order, sequenced so that each phase produces something demonstrable before moving on — never leave a long stretch with nothing working end-to-end.

Phase	Focus	Exit Criteria
Phase 1	Auth, workspace/board CRUD, basic REST APIs	Can register, log in, create a workspace, create a board with columns and cards, all via REST, verified with a REST client — no real-time yet
Phase 2	Frontend CRUD + drag-and-drop (no live sync yet)	Board renders from the API; drag-and-drop reorders cards and persists via REST; single-user experience is solid before adding multi-user complexity
Phase 3	WebSocket/STOMP integration for card moves	Two browser sessions see card moves live; this is the first "real-time" milestone and worth demoing early
Phase 4	Presence, heartbeat, editing indicators	Online/offline list updates live; heartbeat-based offline detection works even on a silent disconnect; soft-lock indicators appear correctly
Phase 5	Comments, typing indicators, notifications	Comment threads work with live typing indicators; notifications generate and deliver in real time
Phase 6	Activity log, permissions, optimistic concurrency polish	Activity feed is complete and readable; role checks enforced server-side; version-conflict handling is graceful, not just functional
Phase 7	Dockerization, testing pass, deployment, documentation	Whole stack runs via a single Docker Compose command; core flows have test coverage; a README documents setup and architecture for anyone evaluating the project

Sequencing rationale: real-time features (Phases 3–5) are deliberately built *after* a solid single-user CRUD foundation (Phases 1–2), because debugging real-time bugs on top of an already-shaky data layer is far harder than debugging them on top of one you already trust.

19. Testing Strategy

- **Backend unit tests** for service-layer logic — especially the optimistic concurrency check (§9.4) and position-recalculation logic (§8.3), since these are the most bug-prone areas.
 - **Backend integration tests** for REST endpoints (auth flows, CRUD) using an in-memory or containerized test database.
 - **WebSocket/STOMP integration tests** simulating two connected clients to verify broadcast correctness (e.g., a move from client A is received by client B, and the originator's own optimistic state reconciles cleanly).
 - **Manual multi-tab testing** as an ongoing practice throughout development — literally two browser windows side by side is the cheapest and most revealing test you have for this kind of project.
 - **Manual chaos testing** — using browser dev tools to simulate offline/online transitions and verify reconnect-and-resync behavior (§10.6) and presence timeout behavior (§9.3) actually hold up.
-

20. Deployment & Infrastructure

- **Local development:** Docker Compose spinning up PostgreSQL, the Spring Boot backend, and the React frontend (or the frontend run via its own dev server with a proxy to the backend) as one command.
 - **Environment configuration:** separate configuration profiles for local vs. a deployed environment (database URL, JWT secret, allowed CORS origins) rather than hardcoding values.
 - **Deployment target:** any single-instance host is sufficient (a small VM, a container platform, or a PaaS with WebSocket support) — the architecture explicitly does not require multi-instance scaling for v1, so avoid over-engineering the deployment to match a scale you're not targeting.
 - **Reverse proxy (optional, v2):** Nginx in front of both the API and the WebSocket endpoint if you deploy beyond simple Docker Compose, mainly for TLS termination and clean routing.
-

21. Future Enhancements (Post-v1, Resume-Worthy Additions)

Worth listing explicitly so they don't get lost, but deliberately deferred past v1:

- Redis-backed presence and pub/sub, if you want to demonstrate horizontal scaling of the real-time layer across multiple backend instances.
 - Full-text search across cards and comments.
 - Keyboard-shortcut power-user mode (quick card creation, quick navigation).
 - Audit log filtering (by user, by action type, by date range).
 - Granular, configurable roles beyond Owner/Admin/Member.
 - A richer conflict-resolution UI for optimistic concurrency (e.g., a diff-style view instead of a simple "reload and reapply" prompt).
-

22. Glossary

- **Soft lock:** A visible-but-non-blocking indicator that someone else is editing a resource; a UX signal, not an access control mechanism.
 - **Optimistic concurrency control:** A conflict-detection strategy where writes carry the version they were read at, and are rejected if that version is stale by the time they arrive — as opposed to locking a resource up front.
 - **Ephemeral event:** A real-time event that is broadcast but never persisted to the database (e.g., typing indicators, presence pings).
 - **Durable event:** A real-time event that corresponds to a persisted state change (e.g., a card move, a new comment) — always written to the database and then broadcast.
 - **Cold-start sync:** The initial full-state fetch (via REST) a client performs when first opening a board or reconnecting after a dropped connection, as opposed to relying on incrementally replayed events.
 - **Topic (STOMP):** A named channel clients subscribe to in order to receive a scoped subset of real-time events (e.g., a specific board's topic, or a user's personal notification topic).
-

End of document. This PRD is meant to be a living reference — update it as design decisions evolve during actual implementation, particularly §8 (Data Model) and §11 (Event Catalog),

which are the two sections most likely to need refinement once you're hands-on with the code.